

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Experiments (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Tejaswi Reddy K	Reg. No.:	192371036
Section / Batch:	CS-BIOSCI/23	Date:	13-08-2026

Instructions to Students

- Write algorithm and execute code both experiments.
- Each questions 50 Marks. Total: 100 Marks.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Description	Questions	Total Marks
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1 – Q2	100

SECTION A – Experiments

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KOMMURU TEJASWI REDDY

19227104602

QUESTIONS

Q1

Divide and Conquer

30 marks

Q2

Divide and Conquer

30 marks

2/2 answered

Q1 Divide and Conquer

50 marks

Analyze the performance of Quick Sort under different partitioning schemes such as Lomuto and Hoare. Record execution time and number of swaps. Compare results and interpret differences. Justify which partitioning scheme provides better efficiency.

Your Code

```
1 import time
2 import random
3 import sys
4
5 sys.setrecursionlimit(5000)
6
7 class QuickSortAnalysist:
8     def __init__(self):
9         self.swaps = 0
10
11     def reset(self):
12         self.swaps = 0
13
14     def lomuto_partition(self, arr, low, high):
15         pivot = arr[high]
16         i = low - 1
17         for j in range(low, high):
18             if arr[j] <= pivot:
19                 i += 1
20                 arr[i], arr[j] = arr[j], arr[i]
```

Input

stdin input

Output

Array size: 1000
Lomuto Partition:
Execution time: 0.00000 seconds
Number of Swaps: 3718
Hoare Partitioning:

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KOMMURU TEJASWI REDDY

19227104602

```
21         for j in range(low, high):
22             if arr[j] <= pivot:
23                 i += 1
24                 arr[i], arr[j] = arr[j], arr[i]
25                 self.swaps += 1
26         arr[i + 1], arr[high] = arr[high], arr[i + 1]
27         self.swaps += 1
28         return i + 1
29
30     def quick_sort_lomuto(self, arr, low, high):
31         if low < high:
32             p = self.lomuto_partition(arr, low, high)
33             self.quick_sort_lomuto(arr, low, p - 1)
34             self.quick_sort_lomuto(arr, p + 1, high)
35
36     def hoare_partition(self, arr, low, high):
37         pivot = arr[low]
38         i = low - 1
39         j = high + 1
40         while True:
41             i += 1
42             while arr[i] <= pivot:
43                 i += 1
44
45             j -= 1
46             while arr[j] > pivot:
47                 j -= 1
48
49             if i >= j:
50                 return j
```

Activate Windows

Go to Settings to activate Windows.

SIMATS Online Lab

CSA6021-CO3-A11 - Classroom

Aak Gemini

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1190

80

☆

📄

📁

School

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KOMMURU TEJASWI REDDY

19237103680

```
43         j -= 1
44
45         if i == j:
46             return j
47
48         arr[i], arr[j] = arr[j], arr[i]
49         self.swaps += 1
50
51     def quick_sort_hoare(self, arr, low, high):
52         if low < high:
53             p = self.hoare_partition(arr, low, high)
54             self.quick_sort_hoare(arr, low, p)
55             self.quick_sort_hoare(arr, p + 1, high)
56
57     def run_experiment(size=1000, seed=42):
58         random.seed(seed)
59         original_data = [random.randint(0, 100000) for _ in range(size)]
60
61         analysis = QuickSortAnalysis()
62
63         # Lomuto
64         lomuto_data = list(original_data)
65         analysis.reset()
66         start = time.time()
67         analysis.quick_sort_lomuto(lomuto_data, 0, len(lomuto_data) - 1)
68         lomuto_time = time.time() - start
69         lomuto_swaps = analysis.swaps
70
71         # Hoare
72         hoare_data = list(original_data)
73         analysis.reset()
74         start = time.time()
75         analysis.quick_sort_hoare(hoare_data, 0, len(hoare_data) - 1)
76         hoare_time = time.time() - start
77         hoare_swaps = analysis.swaps
78
79         print(f"Array size: {size}")
80         print("Lomuto Partition:")
81         print(f"Execution time: {lomuto_time:.6f} seconds")
82         print(f"Number of swaps: {lomuto_swaps}")
83         print()
84         print("Hoare Partition:")
85         print(f"Execution time: {hoare_time:.6f} seconds")
86         print(f"Number of swaps: {hoare_swaps}")
87
88 if __name__ == "__main__":
89     run_experiment(10000)
```

Activate Windows

Go to Settings to activate Windows.

Submitted

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KOMMURU TEJASWI REDDY

19237103680

```
87     analysis.quick_sort_lomuto(lomuto_data, 0, len(lomuto_data) - 1)
88     lomuto_time = time.time() - start
89     lomuto_swaps = analysis.swaps
90
91     # Hoare
92     hoare_data = list(original_data)
93     analysis.reset()
94     start = time.time()
95     analysis.quick_sort_hoare(hoare_data, 0, len(hoare_data) - 1)
96     hoare_time = time.time() - start
97     hoare_swaps = analysis.swaps
98
99     print(f"Array size: {size}")
100    print("Lomuto Partition:")
101    print(f"Execution time: {lomuto_time:.6f} seconds")
102    print(f"Number of swaps: {lomuto_swaps}")
103    print()
104    print("Hoare Partition:")
105    print(f"Execution time: {hoare_time:.6f} seconds")
106    print(f"Number of swaps: {hoare_swaps}")
107
108 if __name__ == "__main__":
109     run_experiment(10000)
```

Activate Windows

Go to Settings to activate Windows.

SIMATS Online Lab
CSA6021-CO3-A11 - Classroom
Ask Gemini
Not secure simatslab.siveetha.com/practice_app/classactivity/attempt?assignment_id=1190
School

SIMATS Online Programming Lab
DAA PROGRAMMING
KOMMURI TEJASWI REDDY
1923702490

QUESTIONS

Q1
Divide and Conquer
20 marks

Q2
Divide and Conquer
30 marks

2/2 answered

Q2: Divide and Conquer 30 marks

Analyze the performance of Merge Sort by implementing the algorithm on datasets with different input sizes and recording execution time for each case. Plot the results using graphs to visualize time complexity trends. Interpret how the divide and conquer strategy ensures consistent $O(n \log n)$ performance regardless of input distribution. Justify why Merge Sort is preferred for stable sorting applications involving large datasets.

Your Code

```

1 input_size
2 input_random
3
4 def merge_sort(arr):
5     if len(arr) <= 1:
6         return arr
7
8     mid = len(arr) // 2
9     left = merge_sort(arr[:mid])
10    right = merge_sort(arr[mid:])
11
12    return merge(left, right)
13
14 def merge(left, right):
15     result = []
16     i = j = 0
17
18     while i < len(left) and j < len(right):
19         if left[i] <= right[j]:
20             result.append(left[i])
21             i += 1
22         else:

```

Input

Output

```

Size: 100, Time: 0.0000s
Size: 1000, Time: 0.0000s
Size: 10000, Time: 0.0000s
Size: 100000, Time: 0.0000s
Size: 1000000, Time: 0.0000s
Size: 10000000, Time: 0.0000s

```

Activate Windows
Go to Settings to activate Windows.

SIMATS Online Lab
CSA6021-CO3-A11 - Classroom
Ask Gemini
Not secure simatslab.siveetha.com/practice_app/classactivity/attempt?assignment_id=1190
School

SIMATS Online Programming Lab
DAA PROGRAMMING
KOMMURI TEJASWI REDDY
1923702490

```

21     result.append(left[i])
22     i += 1
23     else:
24         result.append(right[j])
25         j += 1
26
27     result.extend(left[i:])
28     result.extend(right[j:])
29     return result
30
31 # Performance Testing
32 sizes = [100, 1000, 10000, 100000, 1000000, 10000000]
33 execution_times = []
34
35 for size in sizes:
36     test_data = [random.randint(0, 1000000) for _ in range(size)]
37     start_time = time.time()
38     merge_sort(test_data)
39     end_time = time.time()
40     execution_times.append(end_time - start_time)
41     print(f"Size: {size}, Time: {end_time - start_time:.3f}s")
42
43 # Plotting the results
44 fig = plt.figure(figsize=(10, 6))
45 plt.plot(sizes, execution_times, marker='b', linestyle='-', color='b')
46 plt.title('Merge Sort Performance Analysis')
47 plt.xlabel('Input Size (n)')
48 plt.ylabel('Execution Time (seconds)')
49 plt.grid(True)
50 plt.show()

```

Submitted

Activate Windows
Go to Settings to activate Windows.

Code of Conduct

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Tejaswi Reddy K

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Experiments	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Marks (Each - 50)
Ex. 1						
Ex. 2						
Overall Total (100)						100 %

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Signature: _____	Signature: _____
Signature: _____		
Date: _____	Date: _____	Date: _____